



**MULUNGUSHI UNIVERSITY
SCHOOL OF ENGINEERING AND TECHNOLOGY
BACHELOR OF SCIENCE IN COMPUTER SCIENCE**

Name: Blessings Moonga

Lab: 2 Task 4.1

Course: Web Systems and Technology

Student Number: 202305090

Lecturer: Mr. Nyerenda

1. Objective

Add a demonstration route that sets a non-sensitive cookie with the `HttpOnly`, `SameSite=Lax` and `Path=` attributes. Call it from the client using credentials: "include" in Fetch, with an exact allowed origin and `Access-Control-Allow-Credentials: true` configured on the server. Capture the `Set-Cookie` header, the stored cookie, and the `Cookie` header on a later request. This is a cookie mechanics demonstration only, not an authentication or login system.

2. Implementation summary

A dedicated route (`GET /api/cookie-demo`) on the Express server sets a cookie named `demoToken` holding an arbitrary placeholder value, using `HttpOnly`, `SameSite=Lax` and `Path=`. CORS is configured with an exact origin (`http://localhost:5500`) rather than a wildcard, plus `Access-Control-Allow-Credentials: true`, since a wildcard origin cannot be combined with credentialed requests. On the client, the route is called with `fetch(url, { credentials: "include" })` so the browser is permitted to send and store the cookie across the cross-origin call between `localhost:5500` (interface) and `localhost:3000` (API).

3. Evidence checklist

Required evidence	What it shows	Figure
Set-Cookie header	Server response sets demoToken with HttpOnly, SameSite=Lax, Path=/ 	Figure 1
Cookie stored in browser	DevTools Application panel lists the stored cookie and its attributes	Figure 2
Cookie on a later request	Browser automatically resends the cookie in a subsequent request	Figure 3

4. Evidence

4.1 Set-Cookie header in the server response

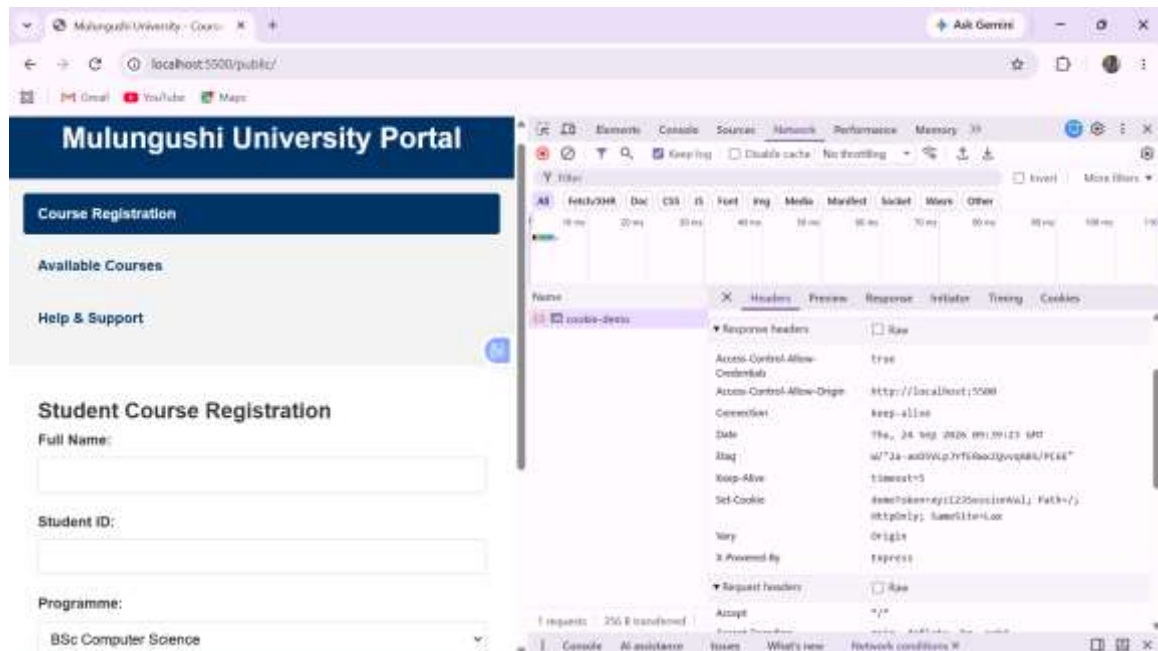


Figure 1. Response headers for GET /api/cookie-demo. Set-Cookie shows demoToken=xyz123SessionVal; Path=/
; HttpOnly; SameSite=Lax, alongside Access-Control-Allow-Origin: http://localhost:5500 and Access-Control-Allow-Credentials: true.

4.2 Cookie stored in the browser

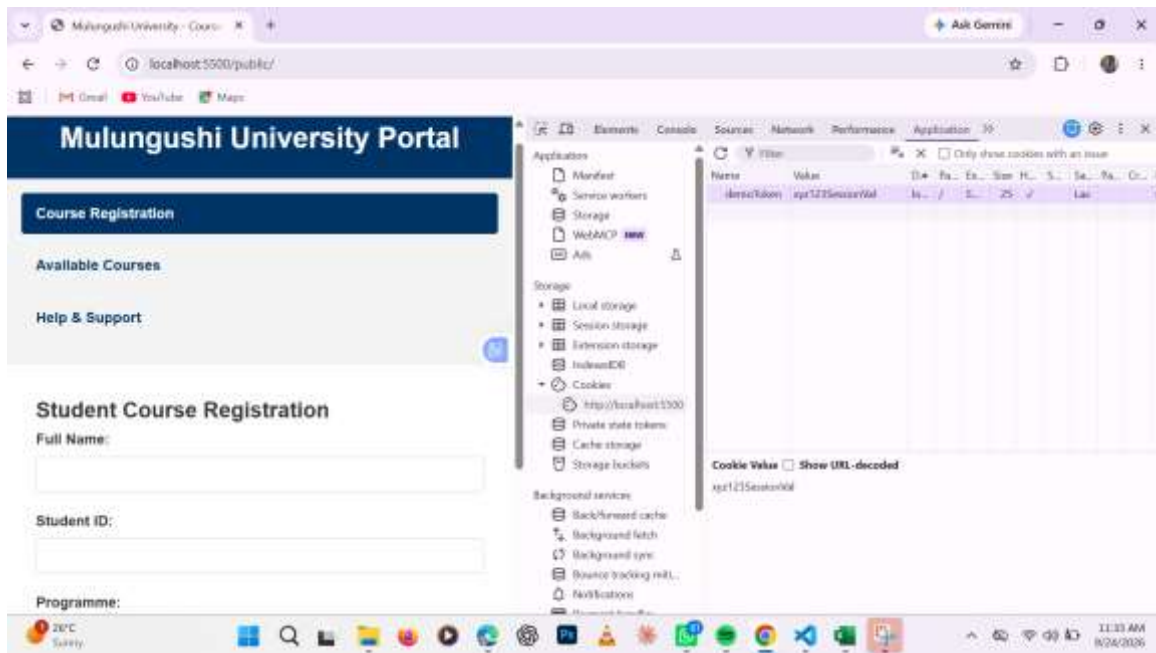


Figure 2. DevTools Application > Cookies panel for localhost:5500, showing demoToken stored with the HttpOnly flag ticked, SameSite set to Lax, and Path set to /.

4.3 Cookie sent automatically on a later request

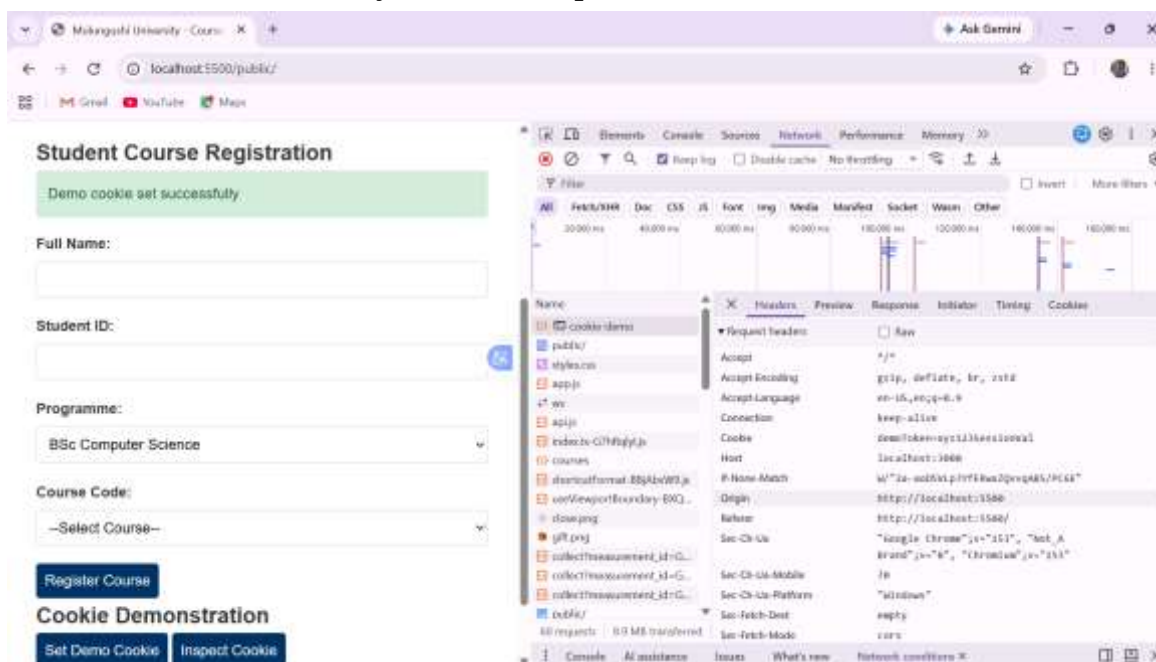


Figure 3. Request headers on a second call to the same endpoint. The browser attaches Cookie: demoToken=xyz123SessionVal automatically, without any client-side code reading or resending it.

4.4 Command-line verification

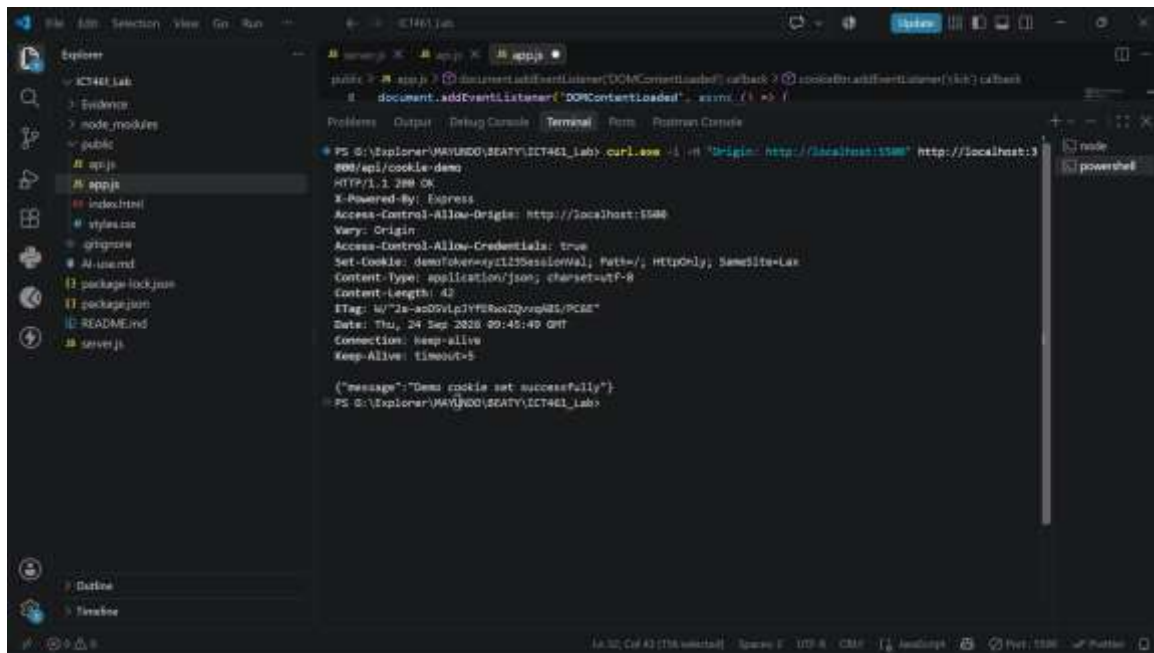


Figure 4. The same endpoint called directly with cURL, passing an Origin header. The identical Set-Cookie and Access-Control-* headers confirm the behaviour originates from the server, not from the browser.

4.5 Credentialed fetch from the console

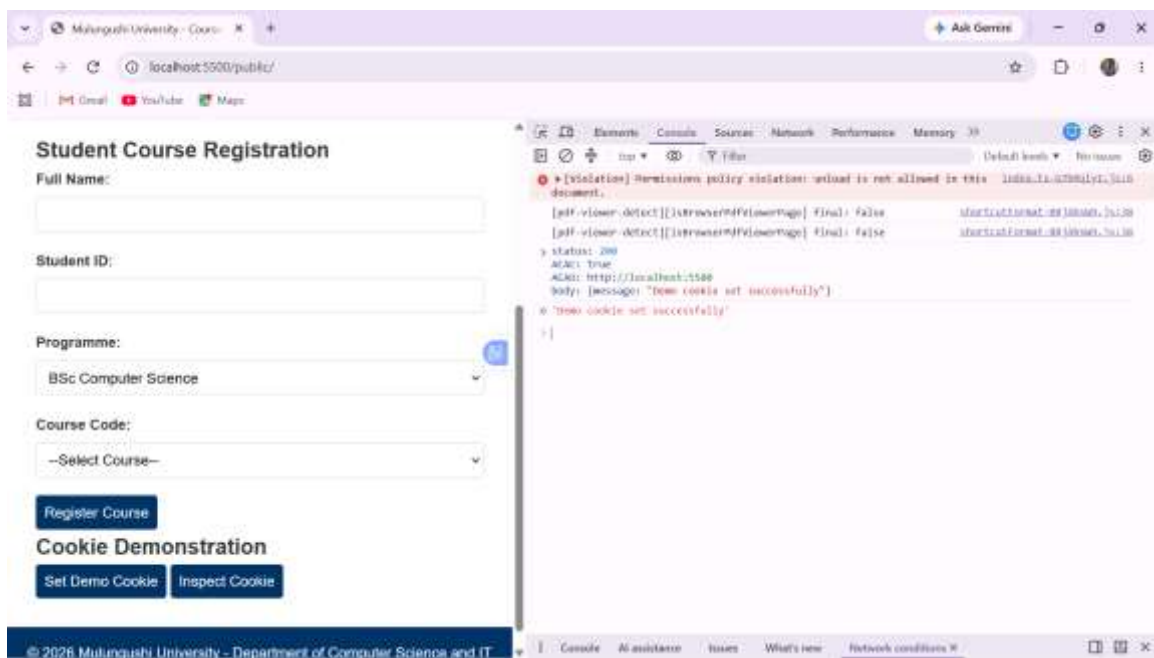


Figure 5. Console output of the fetch call using credentials: "include". Status 200 is returned along with Access-Control-Allow-Credentials: true and Access-Control-Allow-Origin: http://localhost:5500.

4.6 HttpOnly confirmed

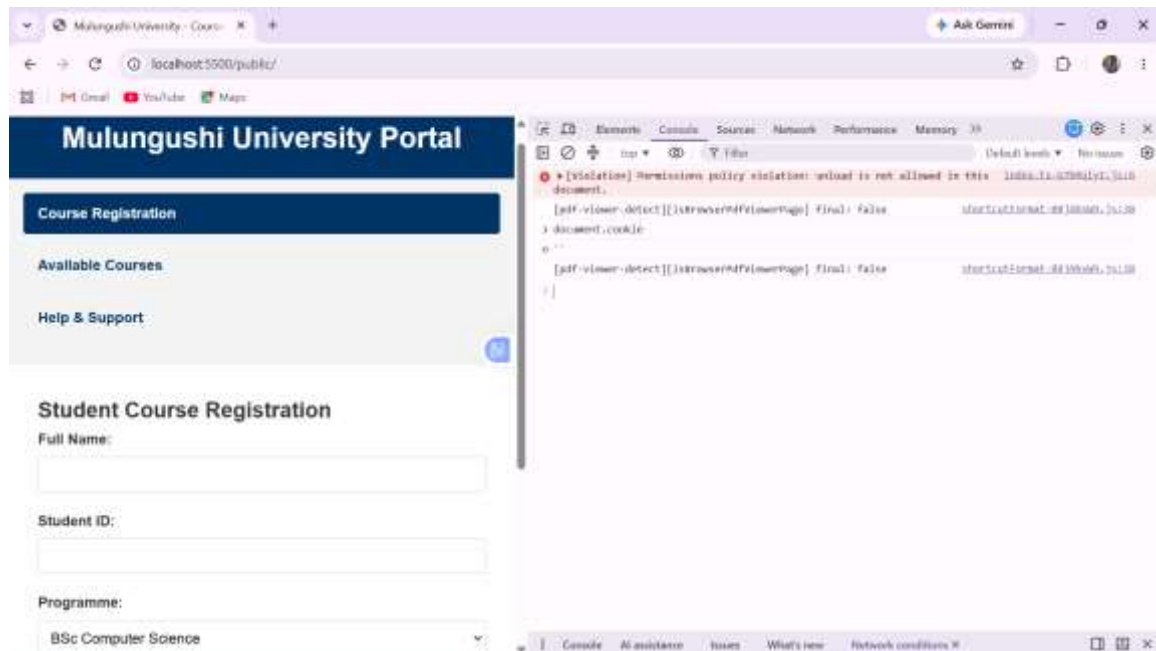


Figure 6. Running `document.cookie` in the console returns an empty string. If `HttpOnly` were not set, the `demoToken` value would appear here and be readable by any script running on the page.

5. Analysis

The CORS side of this task only works because both ends agree: the server names the interface's exact origin and sets `Access-Control-Allow-Credentials: true` (Figures 1 and 4), and the client opts in with `credentials: "include"` (Figure 5). Dropping either side would either stop the cookie from being sent, or have the browser refuse to expose the response to the page's JavaScript.

6. Conclusion

All three pieces of evidence required by the task were captured: the `Set-Cookie` header, the cookie as stored in the browser, and the cookie being resent automatically on a later request. The cookie carries an arbitrary placeholder token rather than any real credential, consistent with this being a mechanics demonstration and not a login system.